MCP

Model Context Protocol

Python
Claude / Claude Desktop
APIs

Why MCP Had to Exist ?

ChatGPT was launched

1 million

100 million

ChatGPT

Totally new kind of way
to interact with machines

ChatGPT can also be used in professional manner.

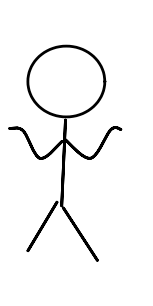
LLM (Large Language Model)

Hi → LLM → Hi, Mayank how are you ?

just a next word/token predictor

Code

Model



→ Lazy

Give me the email for my sick leave,
it send the same as well.

Not only provide me with the code, actually
write and run it on my machine.

Gmail
Slack
Word ONline
Outlook
Drive

Tell us what to do

but rather do the same for us.

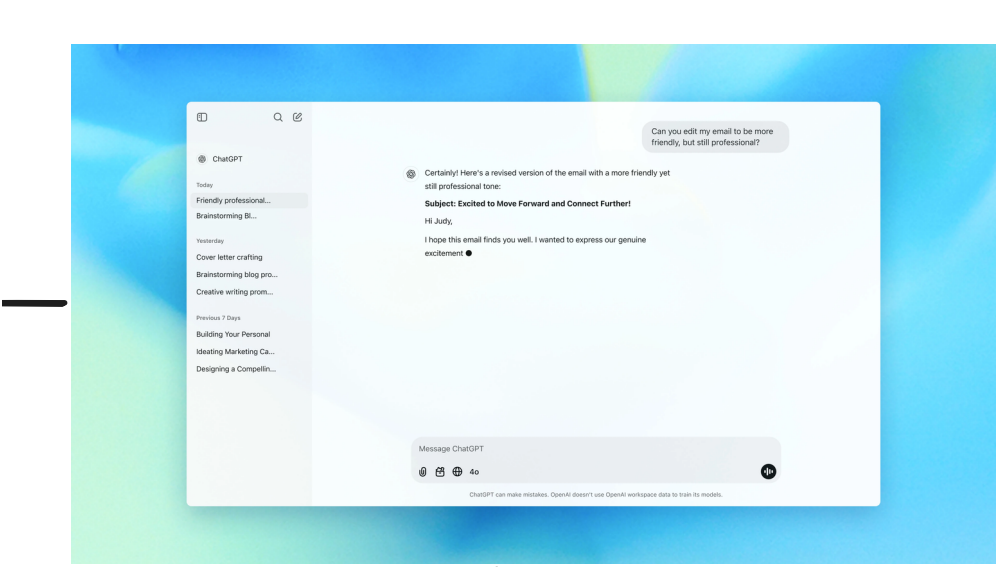
→ LLM →

Context

Context is everything that your LLM/Model
can see when it answers your question

→ ChatGPT

MCP is not about giving our LLM more intelligence, it is about giving it
an ability to see more.



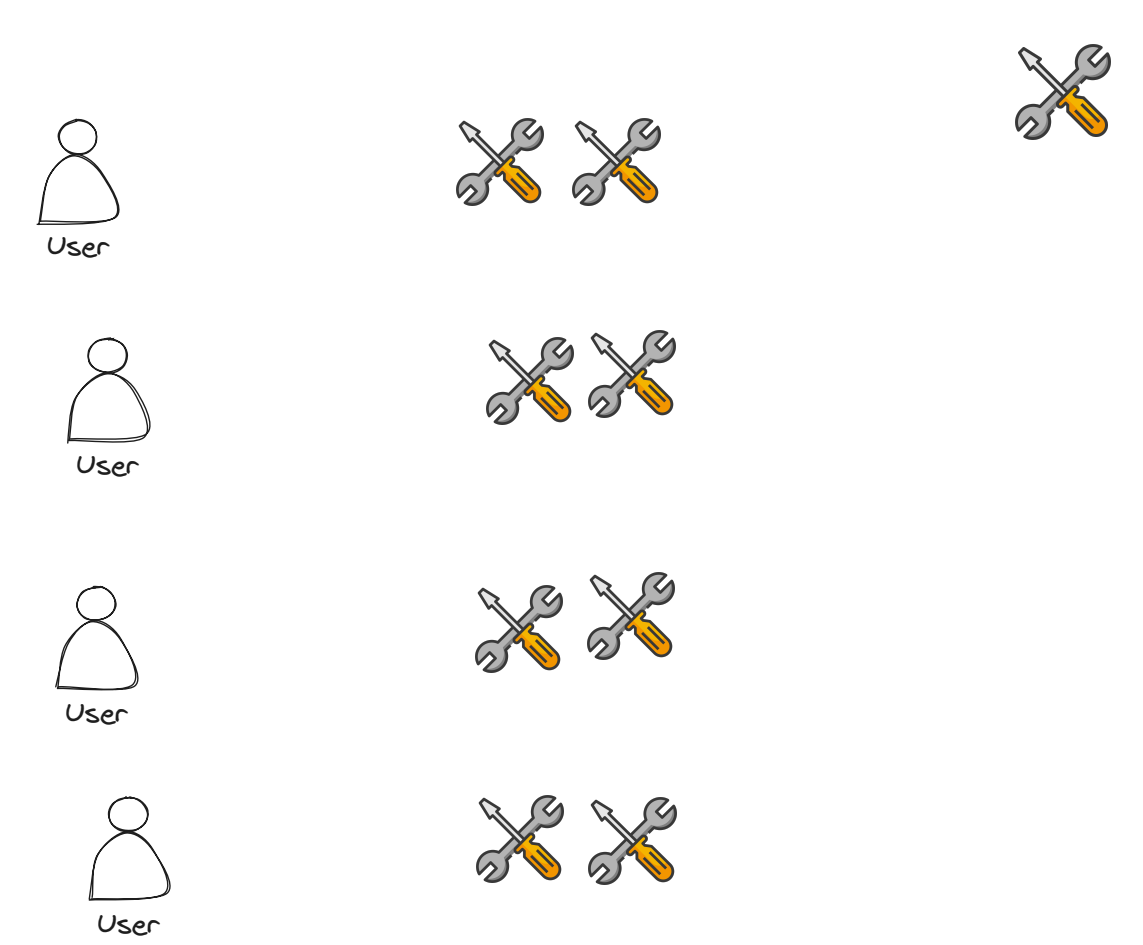
Read msgs ← → Read Emails

Function Call

Web Search

Search (President of Venenzuala)

llm.bind_tool()



User
User
User
User

Tool

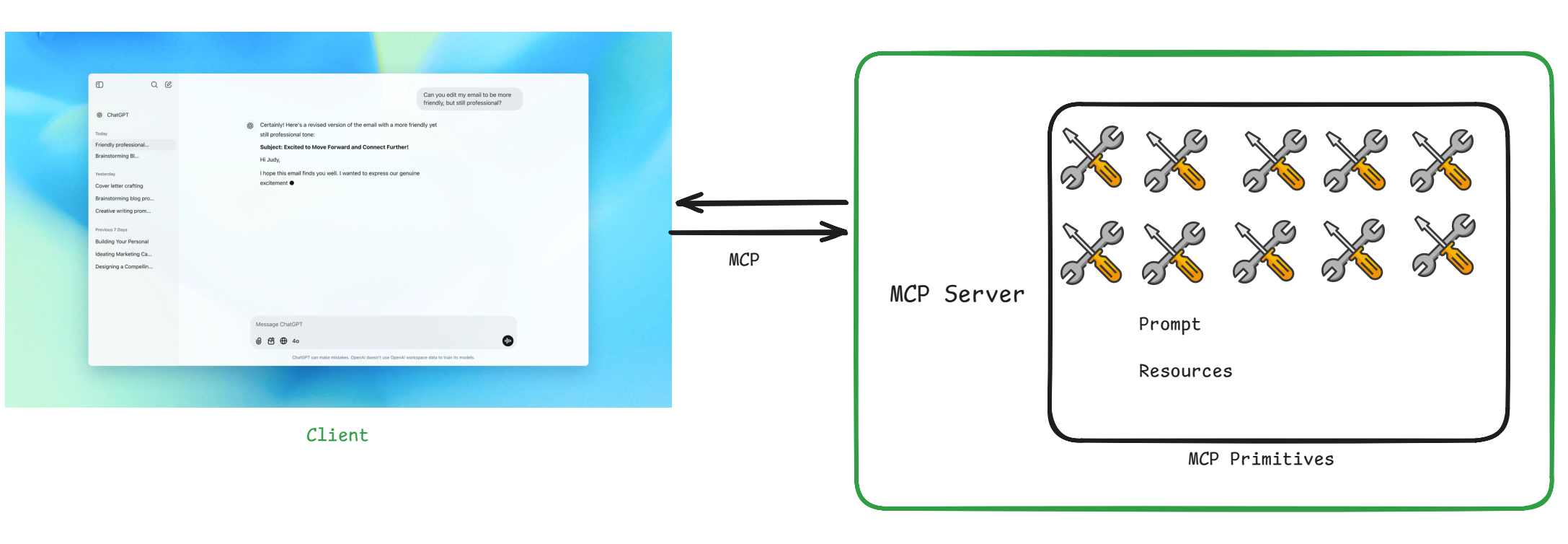
Action → API being called

× 20

4 applications * 6 Actions = 24 Tools

User

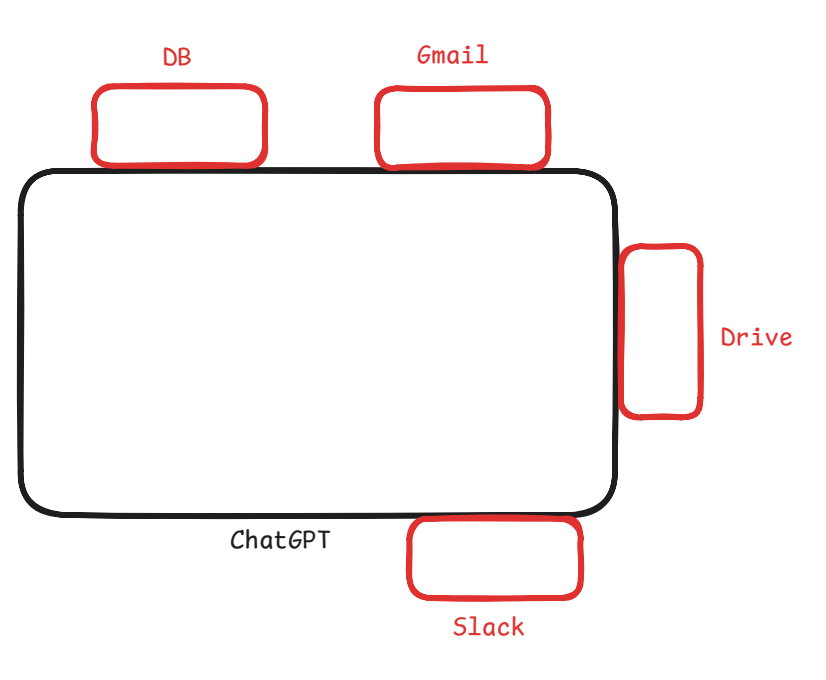
Save Time
↓
Maintain our Tools



Client
MCP
MCP Server
Prompt
Resources
MCP Primitives

APIs

Tools or Actions



DB
Gmail
Drive
ChatGPT
Slack

MCP Components

MCP Client ⇄ MCP Server

+ MCP Client → MCP Server

STDIO

Streamable HTTP

HTTP + SSE
(depreciated)

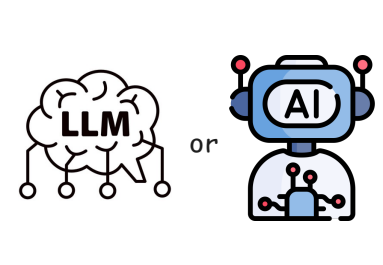
LLM <---> MCP Client ——Protocol——> MCP Server

Host

tools
prompts
resources

Claude Desktop
Agentic
Cursor

Before & After MCP



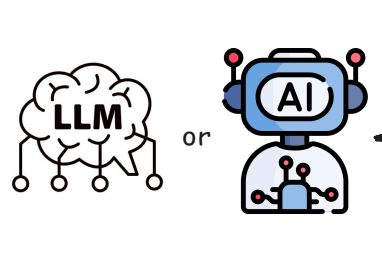
⇄ Gmail Send API

1. Super easy to define.
2. Easily Understandable.

1. Duplicate Code by everyone or in every project.

2. Reduntant + Prone to failure or change

3. Just a wrapper around APIs



Gmail MCP Server

MCP Server

Resources
Prompt
Tools

Gmail API

send email
draft email
read email

1. Easy to define and access.
2. Scalable
3. No Reduntant Code.
4. Access to all common tools and applications.
5. Less Prone to Failure.

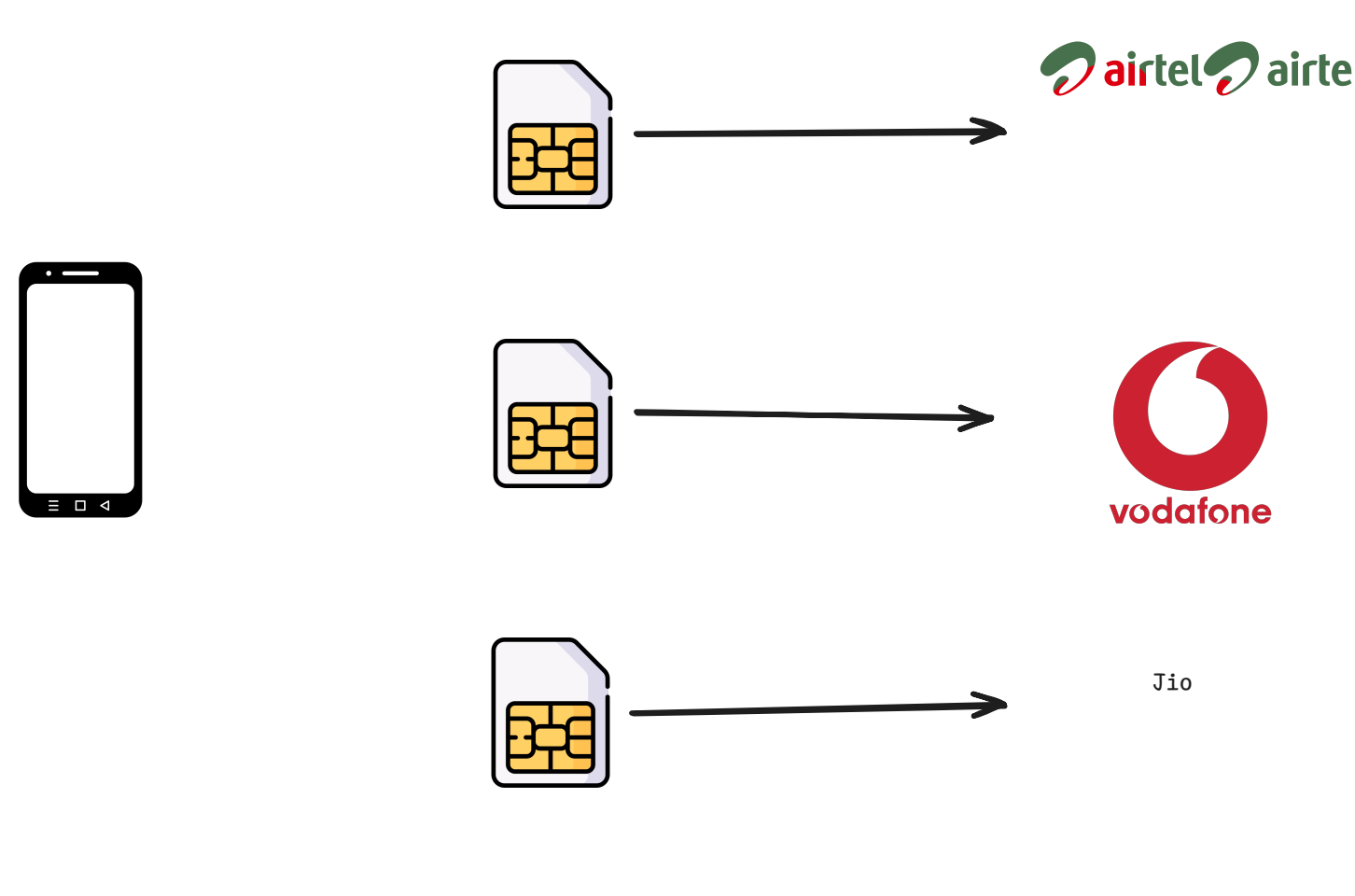
1. Little Complex Initially.
2. Many Servers can run locally.
3. Less Idea about the implementation.
(This was actually the goal as well).

MCP Architecture

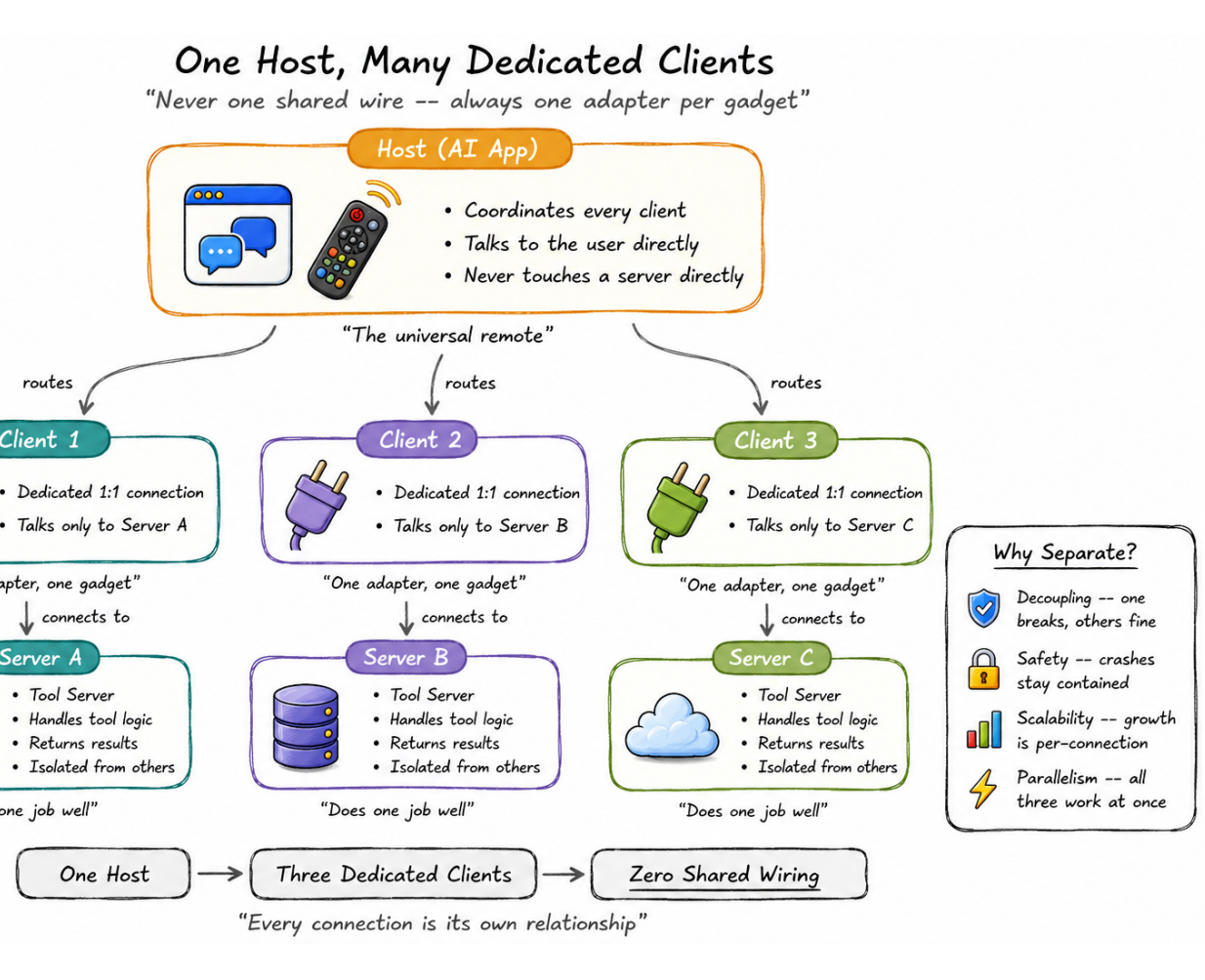
Host

Client

Server



airtel
airtel
vodafone
Jio



One Host, Many Dedicated Clients
"Never one shared wire -- always one adapter per gadget"
Host (AI App)
Coordinates every client
Talks to the user directly
Never touches a server directly
"The universal remote"
routes
routes
routes
Client 1
Dedicated 1:1 connection
Talks only to Server A
"One adapter, one gadget"
connects to
Client 2
Dedicated 1:1 connection
Talks only to Server B
"One adapter, one gadget"
connects to
Client 3
Dedicated 1:1 connection
Talks only to Server C
"One adapter, one gadget"
connects to
Server A
Tool Server
Handles tool logic
Returns results
Isolated from others
"Does one job well"
Server B
Tool Server
Handles tool logic
Returns results
Isolated from others
"Does one job well"
Server C
Tool Server
Handles tool logic
Returns results
Isolated from others
"Does one job well"
Why Separate?
Decoupling -- one breaks, others fine
Safety -- crashes stay contained
Scalability -- growth is per-connection
Parallelism -- all three work at once
One Host
Three Dedicated Clients
Zero Shared Wiring
"Every connection is its own relationship"

HOST

HOST → Server

Claude Desktop
VSCODE
Cursor
AI or Chatbot
Codex

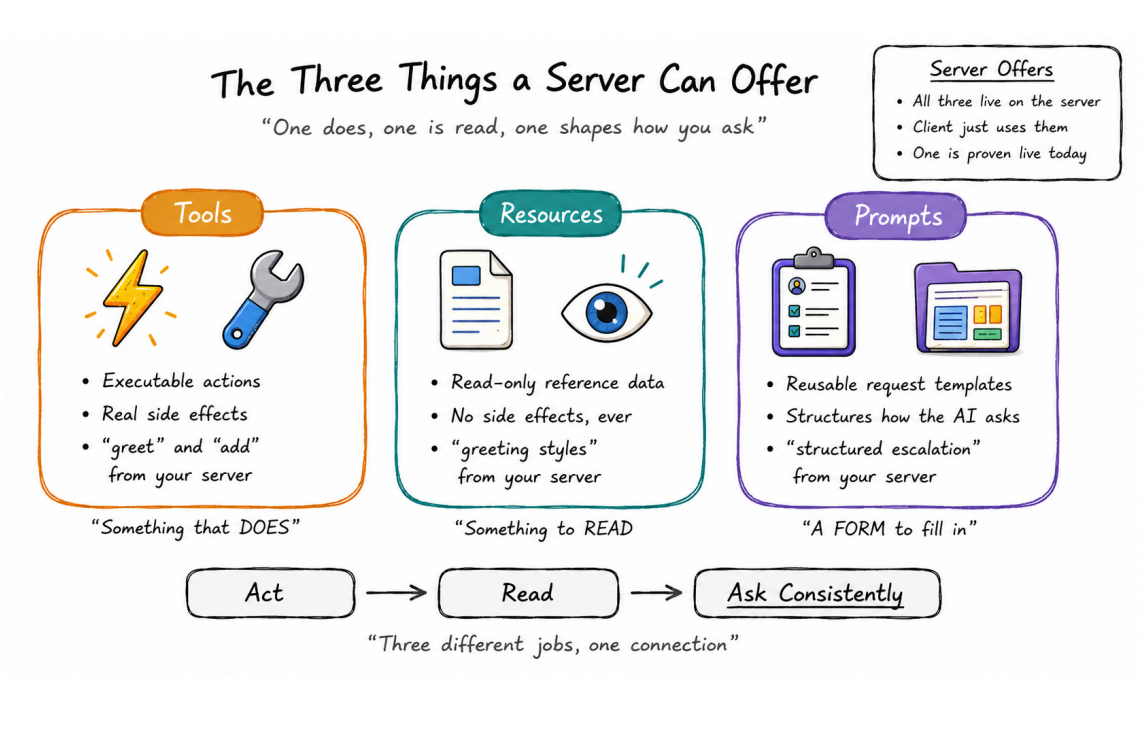
Applications/Tools like Github
Gmail, Drive, Slack, JIRA,
your Own awesome MCP Server
Canva

HOST + MCP Client → Server

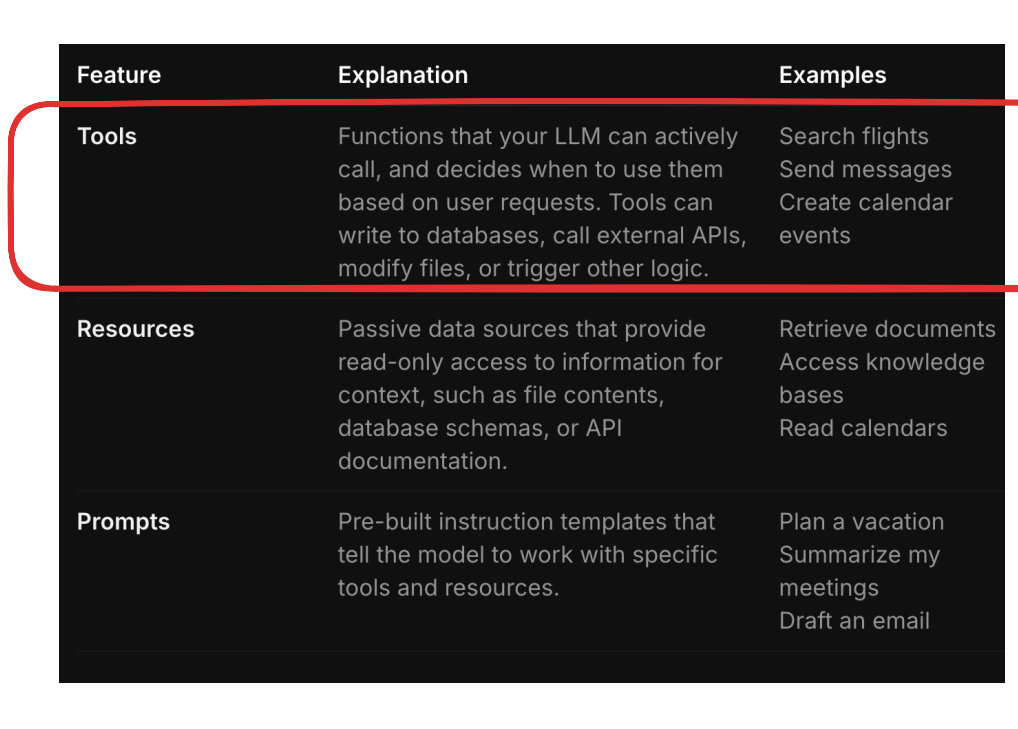
What it is	The AI application a person actually talks to — receives the question, decides if outside help is needed, coordinates everything after that
Where it was, just now	Inspector's own interface, standing in for one
Real examples	Claude Desktop, Cursor, a custom chatbot
The one rule that matters	The host never talks to a server directly — it delegates entirely

MCP
Client → Langchain
MCP Server

SERVER



The Three Things a Server Can Offer
"One does, one is read, one shapes how you ask"
Server Offers
All three live on the server
Client just uses them
One is proven live today
Tools
Executable actions
Real side effects
"greet" and "add" from your server
"Something that DOES"
Resources
Read-only reference data
No side effects, ever
"greeting styles" from your server
"Something to READ"
Prompts
Reusable request templates
Structures how the AI asks
"structured escalation" from your server
"A FORM to fill in"
Act
Read
Ask Consistently
"Three different jobs, one connection"



Feature	Explanation	Examples
Tools	Functions that your LLM can actively call, and decides when to use them based on user requests. Tools can write to databases, call external APIs, modify files, or trigger other logic.	Search flights Send messages Create calendar events
Resources	Passive data sources that provide read-only access to information for context, such as file contents, database schemas, or API documentation.	Retrieve documents Access knowledge bases Read calendars
Prompts	Pre-built instruction templates that tell the model to work with specific tools and resources.	Plan a vacation Summarize my meetings Draft an email

Delete Entry

Schema for
Data

Instructions

Servers have URL, they are kind of like any other server so
they can have auth and everything else

Any host can connect to the server and a server ideally or typically supports the
complete way of giving access to MCP primitives (Tools, resources and Prompts)